

অপারেটিং সিস্টেমের ভূমিকা

(Introduction to Operating Systems)

প্রফেসর চেস্টার রেবিয়ার

(Prof. Chester Rebeiro)

কম্পিউটার সায়েন্স এন্ড ইঞ্জিনিয়ারিং

(Department of Computer Science and Engineering)

ইন্ডিয়ান ইনস্টিটিউট অফ টেকনোলজি (Indian Institute of Technology), মাদ্রাজ

সপ্তাহ – 01 বক্তৃতা- 03

প্রোগ্রাম থেকে প্রসেস

নমস্কার। আজকের ক্লাসে আমরা প্রসেস নামে পরিচিত একটি অপারেটিং ধারণার খুব সংক্ষিপ্ত ভূমিকা রাখছি।

(স্লাইডটাইপ পড়ুন: 00:23)

সুতরাং, আমরা যে বিষয়গুলি আলোচনা করবো, তা হল প্রোগ্রাম থেকে প্রসেস, মেমরি ম্যাপ, একটি সিস্টেম কল, ফাইল এবং মূলত অপারেটিং সিস্টেমের গঠন।

(স্লাইডটাইপ পড়ুন: 00:37)

C তে লেখা এই বিশেষ প্রোগ্রামটি বিবেচনা করুন। এই প্রোগ্রামটি পর্দায় "হ্যালো ওয়ার্ল্ড" প্রিন্ট করে। এই প্রোগ্রামটি কম্পাইল এবং চালানোর জন্য, প্রথমে আমরা একটি কম্পাইলার যেমন gcc এবং একটি সি কোডের নাম উল্লেখ করতে চাই এই ক্ষেত্রে যেমন হ্যালো ডট সি এবং আমরা যা পাই তা এক্সিকিউটেবল, এই ক্ষেত্রে এটি ডট আউট। এই এক্সিকিউটেবলটি হার্ড ডিস্কে সংরক্ষিত হয়। এই নির্দিষ্ট প্রোগ্রামটি চালানোর জন্য আমরা একটি কমান্ড উল্লেখ করি যেমন ডট স্ল্যাশ এ ডট আউট (./a.out) এবং এর ফলে RAM-র মধ্যে একটি প্রসেস তৈরী হয়। সুতরাং, এই প্রসেসটি মূলত এ ডট আউট প্রোগ্রাম যেটা চলছে এবং যা RAM এ উপস্থিত।

(স্লাইডটাইপ পড়ুন: 01:18)

সংজ্ঞা হিসাবে আমরা বলতে পারি, প্রসেস হল একটি চালু প্রোগ্রাম যা RAM থেকে কার্যকর হয় এবং মূলত বিভিন্ন বিভাগ যেমন এক্সিকিউটেবল নির্দেশাবলী, স্ট্যাক, হীপ এবং একটি লুকানো বিভাগ যা স্টেট নামে পরিচিত। সুতরাং, এই স্টেট আসলে অপারেটিং সিস্টেম দ্বারা পরিচালিত হয় এবং এতে বিভিন্ন জিনিস রয়েছে যেমন রেজিস্টার, খোলা ফাইলের তালিকা, প্রসেস, সম্পর্কিত প্রসেসগুলোর তালিকা ইত্যাদি। সুতরাং, আজকের ক্লাসে আমরা বিশদভাবে দেখব এই প্রসেসে কীকি থাকে এবং এটি কিভাবে পরিচালিত হয়।

(স্লাইডটাইপ পড়ুন: 01:55)

সুতরাং, আসুন আমরা একটি খুব সহজ উদাহরণ দেখি। সুতরাং, এটি একটি প্রোগ্রাম এবং এই একটি প্রসেস যা তৈরি হয় যখন এই প্রোগ্রামটি চালানো হয়। এখন প্রসেসে বিভিন্ন বিভাগ আছে যেমন টেক্সট (text), ডেটা (data), হীপ (heap) এবং স্ট্যাক (stack)। এখন এই প্রোগ্রামের বিভিন্ন অংশ যখন এক্সিকিউট হয় তখন প্রসেসের বিভিন্ন অংশগুলোতে ম্যাপ হয়। উদাহরণস্বরূপ, মেইন ফাংশনের সাথে জড়িত সমস্ত ইন্সট্রাকশনের সঙ্গে প্রসেসের টেক্সট বিভাগের ম্যাপ করা হবে। অনুরূপভাবে, অন্যান্য ফাংশনগুলি এখানে যেমন ফ্যাক্ট ফাংশন হিসাবে এখানে জড়িত ইন্সট্রাকশনগুলিও টেক্সট বিভাগের সঙ্গে ম্যাপ হবে। এখন গ্লোবাল ডাটা এবং স্ট্যাটিক ডেটা প্রক্রিয়াটির ডেটা সেকশনে ম্যাপ করা হয়। সুতরাং, এই বিভাগটি প্রকৃতপক্ষে দুটি অংশে বিভক্ত হয় যথা ইনিশিয়ালইজ এবং নন-ইনিশিয়ালইজ বিভাগ। তৃতীয় অধ্যায়টি হীপ, এখন যে কোন ডাইনামিকাল (dynamical) মেমরি যেমন m যেটা কিনা malloc ব্যবহার করে ডাইনামিকাল ভাবে বরাদ্দ করা হয় সেটি হীপে তৈরি হয়। এখন চূড়ান্ত অংশটিকে বলা হয় স্ট্যাক, যা সমস্ত লোকাল ভেরিয়েবল যেমন n এবং m এবং ফাংশন ইনভোকেশন সম্পর্কে তথ্য রয়েছে। উদাহরণস্বরূপ, এই ক্ষেত্রে আমাদের একটি রিকারসিভ (recursive) ফাংশন আছে যা চালু করা হচ্ছে। সুতরাং, এই সমস্ত তথ্য স্ট্যাকের মধ্যে উপস্থিত। এখন, একটি প্রসেসের মেমরি ম্যাপের সর্বাধিক সীমা সর্বাধিক MAXSIZE বলা হয়। সাধারণত অপারেটিং সিস্টেমের মধ্যে আজকাল ব্যবহৃত প্রসেসগুলোর MAXSIZE অপারেটিং সিস্টেম দ্বারা নির্ধারিত হয়। উদাহরণস্বরূপ একটি 32 বিট লিনাক্স অপারেটিং সিস্টেমের মধ্যে, প্রত্যেকটি প্রসেসের সর্বাধিক আকার 0xc এ নির্ধারিত হয় 0000000 এর পরে। xv6 অপারেটিং সিস্টেম যা আমরা এই কোর্সের জন্য খুঁজছি, প্রসেসের সর্বাধিক আয়তন 0x80000000 ঠিকানায় স্থায়ী।

(স্লাইডটাইপ পড়ুন: 04:07)

তাই, আমরা যা দেখেছি তা হল যে যদি প্রতিটি প্রক্রিয়া, একটি প্রোগ্রাম অর্থাৎ একটি প্রোগ্রাম যেটা এক্সিকিউট হচ্ছে একটি এলাকার মধ্যে ম্যাপ করা হয় যা 0 থেকে শুরু হয় এবং MAXSIZE এ শেষ হয়। সুতরাং, প্রসেসের এই MAXSIZE এর পেছনে কি আছে ? সাধারণত কার্নেল (Kernel) বা অপারেটিং সিস্টেমকে মেমরির MAXSIZE থেকে সর্বোচ্চ সীমা পর্যন্ত ম্যাপ করা হয়। টেক্সটের মত সম্পূর্ণ বিষয় অর্থাৎ অপারেটিং সিস্টেমের ইন্সট্রাকশনগুলি, অপারেটিং সিস্টেম ডেটা, ওএস হীপ এবং ডিভাইস মেমরিটি ওএসের এই উপরের অঞ্চলে ম্যাপ

হয়। সুতরাং, সাধারণত কোনও ইউজার প্রোগ্রাম এই নীচের অঞ্চলের কোনও অংশ অ্যাক্সেস করতে পারে যেটা এখানে সবুজ অঞ্চল। সুতরাং, এই বিভাগগুলির কোনটি থেকে প্রকৃতপক্ষে তথ্য পড়তে বা এই বিভাগগুলির অংশে তথ্য লিখতেও সমস্যা হবে না। তবে, প্রসেস কার্নেল মেমরির মধ্যে বিদ্যমান তথ্যটি অ্যাক্সেস করতে পারে না যেটা MAXSIZE এর সীমানার বাইরে। যাইহোক, কার্নেল বা অপারেটিং সিস্টেম যা এই উর্ধ্ব প্রান্ত থেকে এক্সিকিউট করা হয় সেটি অঞ্চলের যে কোনও অংশ থেকে ডাটা অ্যাক্সেস করতে পারে যা এই কার্নেল অঞ্চলের পাশাপাশি ইউজার স্পেস অঞ্চলে ডাটা অ্যাক্সেস করতে পারে। এখন কি হবে যখন আমাদের একই সিস্টেমের মধ্যে একাধিক প্রসেস চলছে? সুতরাং, প্রতিটি প্রসেসের নিজস্ব মেমরির ম্যাপ থাকবে, আমরা তার নিজস্ব ইন্ডেক্সন, ডেটা, হীপ এবং স্ট্যাকের তথ্য রাখব, এবং কার্নেল উপাদানও MAX SIZE এর বাইরে উপস্থিত। সুতরাং, আপনি কি দেখছেন যে সিস্টেমের প্রতিটি প্রসেসের কার্নেলটি MAXSIZE থেকে শুরু করবে এবং অতিক্রম করবে। প্রতিটি প্রসেস যেটি এক্সিকিউট হচ্ছে শুধুমাত্র নিম্ন অংশ এবং এই কার্নেল অংশ একই হবে। এই MAXSIZE এর নীচে একটি প্রক্রিয়া থেকে অন্যটি আলাদা। তাহলে এর অর্থ কি? সুতরাং, এর মানে হল যে যখন আপনি একটি প্রক্রিয়া চালান এবং তারপর অন্য প্রক্রিয়াটি সম্পাদন করেন, তখন MAX SIZE এর উপরের অঞ্চলগুলি একই হতে চলেছে, যখন MAXSIZE এর নিচে অঞ্চলগুলি এক প্রক্রিয়ায় অন্য একটি পরিবর্তন করতে যাচ্ছে। সুতরাং, আমরা উল্লেখ করি যে ব্যবহারকারী প্রোগ্রামগুলি কার্নেল স্থানটিতে যেকোন তথ্য অ্যাক্সেস করতে পারবে না। সেক্ষেত্রে কিভাবে ব্যবহারকারী প্রোগ্রাম প্রকৃতপক্ষে অপারেটিং সিস্টেম আহ্বান করে?

(স্লাইডটাইপ পড়ুন: 06:40)

সুতরাং, বিশেষ আহ্বান ফাংশন আছে যা অপারেটিং সিস্টেম সমর্থন করে, এগুলি সিস্টেম কল নামে পরিচিত। সুতরাং একটি সিস্টেম কল ফাংশন একটি সেট যা OS সমর্থন এবং একটি ইউজার প্রসেস কার্নেলের মধ্যে অন্যান্য সম্পদের জন্য তথ্য পেতে বা হার্ডওয়্যার অ্যাক্সেস করার জন্য কোন সিস্টেম কল আহ্বান করতে পারে। সুতরাং, যখন একটি সিস্টেম কল আহ্বান করা হয়; এটি সাধারণত একটি ইউজার মোডে চলতে থাকা একটি প্রসেস যা কার্নেল মোড বা একটি বিশেষাধিকার মোডে পরিচিত হয় যা কার্নেল বা অপারেটিং সিস্টেমকে আসলেই এক্সিকিউট করতে সক্ষম করে। যখন সিস্টেম কলটি কার্যকর করা হয় তখন ব্যবহারকারীর প্রসেস তার কার্যকারিতা পুনরায় চালু করবে যেখানে এটি একসময় বন্ধ হয়ে যায়।

(স্লাইডটাইপ পড়ুন: 07:25)

সুতরাং, আসুন printf স্টেটমেন্টের একটি উদাহরণ গ্রহণ করি। printf এ আসলে একটি লাইব্রেরী কল হয়। সুতরাং, এটি এই libc তে উপস্থিত রয়েছে এবং এটি বিশেষভাবে কাজ করে ব্যবহারকারীর কাছে কিছু লেখার

জন্য পরিচিত এবং এই ফাংশন তারপর একটি সিস্টেম কল করবে STDOUT প্যারামিটার সহ যার নাম লেখা। সুতরাং, STDOUT একটি বিশেষ প্যারামিটার যা মূলত অপারেটিং সিস্টেমকে বলে দেয় যে printf দ্বারা প্রদত্ত এই স্ট্রিংটি স্ক্রিনের আদর্শ আউটপুটে প্রদর্শন করা উচিত। সুতরাং, লেখাটি হল সিস্টেম কল যা ফাঁদটিকে চালিত করে, এবং এই ফাঁদটি কার্নেল স্পেসের ট্র্যাপ হ্যান্ডলার হিসাবে পরিচিত হবে এবং ট্র্যাপ হ্যান্ডলার একটি ফাংশন চালু করবে যা সিস্টেম কল লেখার সাথে সঙ্গতিপূর্ণ হবে। সুতরাং, এই লেখা সিস্টেম কল আসলে পর্দার উপর str দ্বারা উপলব্ধ স্ট্রিং মুদ্রণের জন্য দায়ী হবে। লেখার সিস্টেম কল সমাপ্ত হওয়ার পরে, এক্সিকিউশন ব্যবহারকারী স্থানে স্থানান্তর করা হয় এবং প্রসেস এক্সিকিউট করতে থাকে।

(স্লাইডটাইপ পড়ুন: 08:40)

একটি স্ট্যান্ডার্ড ফাংশন কল বা প্রসিডিউর কল এবং একটি সিস্টেম কল মধ্যে পার্থক্য কি। এক গুরুত্বপূর্ণ পার্থক্য হচ্ছে, যখন আমরা একটি প্রোগ্রামে একটি ফাংশন আহ্বান করতে চাই অথবা একটি প্রক্রিয়ার মধ্যে আমরা একটি নির্দেশ ব্যবহার করি যেমন একটি CALL নির্দেশনা এটি একটি আদর্শ x64 সমাবেশ নির্দেশনা, এবং এটা ফাংশনকে কল করবে আর ফাংশন কল করার পরে এটি আবার কলিং ফাংশনে ফিরে যায়। যদিও একটি সিস্টেম কলকে ডাকবার জন্য, আমরা TRAP ইন্ট্রাকসন যেমন int 0x80 ব্যবহার করি। সুতরাং এখানে int বলতে এখানে ইনটেরাপ্ট বা সস্টওয়্যার ইনটেরাপ্ট বলা হচ্ছে যার ফলে এটি ব্যবহারকারী স্থান থেকে কার্নেল স্পেসে স্থানান্তরন ঘটে। সুতরাং, ট্র্যাপ (trap) ইন্ট্রাকসনটির ফলে কার্নেলের আহ্বান হয় আর এর ফলে কার্নেলের ইন্ট্রাকসনগুলি এক্সিকিউট হয়। যাইহোক, যখন আমরা স্ট্যান্ডার্ড ফাংশন কল ব্যবহার করি বা কলের প্রসিডিউর ব্যবহার করে পদ্ধতিগত কল ব্যবহার করি তখন ব্যবহারকারীর স্থান থেকে কার্নেল স্পেসের পরিবর্তন বা স্থানান্তর হয় না। সুতরাং, ব্যবহারকারীর স্থানে এক্সিকিউশন অব্যাহত থাকবে। সিস্টেম কল এবং স্ট্যান্ডার্ড প্রসিডিউর কল বা একটি ফাংশন কলের মধ্যে অন্য একটি পার্থক্য হল যে গন্তব্য ঠিকানা বা গন্তব্য ফাংশন একটি রেলোকেটেবল(relocatable)অ্যাড্রেস হতে পারে। সুতরাং, এটি প্রোগ্রাম কম্পাইলের যেকোনো সময় পরিবর্তন হতে পারে। যদিও সিস্টেম কলে যখন একটি ট্র্যাপ নির্দেশ ব্যবহার করা হয় বা প্রসেসর প্রকৃতপক্ষে সিদ্ধান্ত নেয় যে কার্নেল স্পেসের পরবর্তী নির্দেশনা চালু করা উচিত। সুতরাং, এই প্রোগ্রামটি চলমান কি না, কোন অপারেটিং সিস্টেম চলছে এবং এভাবেই তা নির্ধারণ করা যাচ্ছে।

(স্লাইডসময় পড়ুন: 10:30)

সুতরাং, আসলে একটি অপারেটিং সিস্টেম ডিজাইনের গুরুত্বপূর্ণ দিক হল অপারেটিং সিস্টেম কি সিস্টেম কল সমর্থন করে তা? আমরা দেখেছি যে একমাত্র উপায় একটি ব্যবহারকারীর প্রক্রিয়া অপারেটিং সিস্টেমের মধ্যে

একটি নির্দিষ্ট কার্যকারিতা চালু করতে পারে সিস্টেম কল ইন্টারফেসের মাধ্যমে। সুতরাং, প্রশ্ন এখন আসে যদি একজন ব্যক্তি আসলে একটি অপারেটিং সিস্টেম ডিজাইন করে, তাহলে কোন কোন ইন্টারফেসের সিস্টেম কলকে সমর্থন করা উচিত। সুতরাং, একটি সুস্পষ্ট প্রয়োজন হল যে সিস্টেম কল ইন্টারফেসে বেশ কিছু অত্যাধুনিক বৈশিষ্ট্য থাকা উচিত যাতে একটি ব্যবহারকারীর প্রক্রিয়া আসলে খুব সহজেই অপারেটিং সিস্টেমের মধ্যে বেশ কয়েকটি গুরুত্বপূর্ণ কার্যকারিতা ইন্টারফেস বা চালু করতে সক্ষম হয়। যাইহোক, একটি ভিন্ন পদ্ধতি হল অপারেটিং সিস্টেম থেকে প্রয়োজনীয় যে কোনও সহজ সিস্টেম কল ইন্টারফেস এবং অ্যাবস্ট্রাক্ট যেটা দরকার সেটা। সুতরাং, আমরা পরবর্তী স্লাইডে দেখতে পাব, এই একটি বিশেষ উদাহরণ।

(স্লাইডটাইপ পড়ুন: 11:২৭)

আসুন আমরা সিস্টেমের কলগুলির একটি উদাহরণ গ্রহণ করি যে একটি ফাইলগুলি অ্যাক্সেস করার জন্য OS সমর্থন করে। সুতরাং, আমরা জানি যে ফাইলগুলি এমন একটি তথ্য যা রিবুট জুড়ে স্থির হয়, তাই এই তথ্যগুলি হার্ড ডিস্কে সংরক্ষণ করা হয় এবং এগুলি এফওপেন, ক্লোজ, পড়া, লেখা ইত্যাদি ফাংশন ব্যবহার করে পড়া, লেখা বা অ্যাক্সেস করা যায়। প্রত্যেক সময় আমরা একটি ফাইল খুলতে, এটি হার্ড ডিস্ক অ্যাক্সেস করতে হবে, যাতে একটি প্রসেসকে কার্নেলে একটি সিস্টেম কল আহ্বান করতে হবে এবং কার্নেল আসলে হার্ড ডিস্ক বা হার্ড ডিস্ক বাফার অ্যাক্সেস বা অন্যান্য স্টোরেজ মিডিয়াম সাহায্য নিতে হবে ফাইলটি খুলতে এবং প্রসেসটিতে একটি পয়েন্টার ফিরে আসবে। এখন প্রশ্ন আসে কিভাবে অপারেটিং সিস্টেম ডিজাইনার সিদ্ধান্ত নেয় যে ফাইলগুলি অ্যাক্সেস করার জন্য কোন সিস্টেম কল সরবরাহ করা উচিত? সুতরাং, কিছু সুস্পষ্ট জিনিসগুলি যেমন ফাইল খোলার জন্য একটি সিস্টেম কল, ফাইলে পড়া বা লেখার জন্য, তেমনি সিস্টেম কল আছে পরিবর্তন বা সৃষ্টির তারিখ, অনুমতিগুলি সেট এবং যাতে আরও পরিবর্তন করার জন্যও সিস্টেম কল আছে। অপারেটিং সিস্টেম আরও জটিল বা আরো অত্যাধুনিক অপারেশনগুলিকে সমর্থন করতে পারে যেমন একটি ফাইলের মধ্যে একটি বিশেষ অফসেট খোঁজা যায়, যাতে ফাইলগুলির মধ্যে লিঙ্ক করতে সক্ষম হয় এবং আরও অনেক কিছু। সুতরাং, এই প্রয়োজনীয় প্রয়োজনীয়তাগুলি যেগুলি পরিচালনা করার জন্য একটি সিস্টেম কলগুলি সমর্থন করা উচিত। অপর দিকে অপারেটিং সিস্টেম ফাইলটি সম্পর্কে কিছু কিছু তথ্য গোপন করতে সক্ষম হবে। উদাহরণস্বরূপ, সিস্টেম কলগুলি দ্বারা সমর্থিত হওয়া উচিত নয় এমন বিবরণগুলি হল এমন জিনিস যা স্টোরেজ মিডিয়ার বিবরণ যেমন ফাইল সংরক্ষণ করা হয়, উদাহরণস্বরূপ ফাইলটিকে USB ড্রাইভে সংরক্ষণ করা হয় বা হার্ড ডিস্ক বা একটি CD-ROM। এটি আসলে অপারেটিং সিস্টেমের দ্বারা নিখুঁত করা হয় এবং ব্যবহারকারীর প্রসেসটি সহজেই তা জানতে পারবে না যেখানে ফাইলটি সংরক্ষণ করা হয়। অপারেটিং সিস্টেম দ্বারা বিন্যস্ত করা হয় এমন আরেকটি দিক হলো স্টোরেজ মিডিয়াতে সঠিক অবস্থান।

(স্লাইডটাইপ পড়ুন: 13:40)

এখন আমরা দেখব একটি সাধারণ অপারেটিং সিস্টেমের গঠনটি কেমন দেখায়। সুতরাং, অপারেটিং সিস্টেম, আমরা একটি বড় সবুজ ব্লক হিসাবে বিবেচনা করি যাতে একটি অভ্যন্তরীণভাবে নির্মিত বিভিন্ন মডিউল আছে। উদাহরণস্বরূপ, এটি একটি মেমরি ব্যবস্থাপনা মডিউল থাকবে যা সিস্টেমের সমস্ত মেমরি পরিচালনা করে, এখানে একটি সিপিইউ সিডিউলিং ব্লক থাকবে যা ফাইল সিস্টেম ম্যানেজমেন্ট মডিউলের মতো হবে যা ফাইল সিস্টেম যেগুলি হার্ড ডিস্কে বা সিডি-রমগুলিতে রয়েছে তা পরিচালনা করে। সুতরাং, আপনার একটি নেটওয়ার্ক স্ট্যাক আছে যা টিসিপি / আইপি নেটওয়ার্কের পরিচালনা করে এবং কিছু ইন্টার প্রসেস কমিউনিকেশন মডিউল আছে যা একে অপরের সাথে যোগাযোগের প্রক্রিয়াগুলির যত্ন নেবে। সুতরাং, দুইটি গুরুত্বপূর্ণ বিষয় যা এখনো উল্লেখ করা হয়নি, এটি হল সিস্টেম কল ইন্টারফেস, যা ব্যবহারকারী প্রসেসগুলি এই প্রতিটি মডিউলগুলির মধ্যে বৈশিষ্ট্য বা কার্যকারিতাগুলি অ্যাক্সেস করতে দেয়। আরেকটি দিক হলো ডিভাইস ড্রাইভার যা সিস্টেমের মধ্যে হার্ডওয়্যার ডিভাইস এবং অন্যান্য হার্ডওয়্যার রিসোর্সের সাথে যোগাযোগের যত্ন নেবে। সুতরাং, এইগুলি মূলত অপারেটিং সিস্টেমের বিভিন্ন মডিউল।

(স্লাইডসময় পড়ুন: 14:52)

সুতরাং, একটি অপারেটিং সিস্টেমের একক গঠনের মধ্যে, ওএস এ এই সব বিভিন্ন মডিউল একটি একক অ্যাড্রেসেবল কার্নেল স্থানে উপস্থিত রয়েছে, তাই এর মানে এই যে এটি শুধুমাত্র কোড একটি বড় অংশ এবং তাদের সব আপনি একটি বড় প্রোগ্রাম হিসাবে মনে করতে পারেন যেখানে এই সমস্ত মডিউল উপস্থিত রয়েছে। সুতরাং, মেমরি ব্যবস্থাপনা থেকে কোন ফাংশন কল বলে নেটওয়ার্কিং স্ট্যাকের একটি সহজ ফাংশন কল মানে হবে। অনুরূপভাবে নেটওয়ার্কিং স্ট্যাক থেকে ডিভাইস ড্রাইভার অন্য ফাংশন কল হতে হবে। এই মূলত এই ধরনের একধরনের গঠন থাকার সুবিধা, আপনি একটি মডিউল এবং অন্য মডিউলের মধ্যে একটি সরাসরি যোগাযোগ থাকতে পারে। অন্যদিকে, কার্নেল স্থান খুব বড় হয়ে যায়, এবং সেইজন্য, এটা মেনে চলা কঠিন এবং আরো বাগ থাকে। সুতরাং, সাধারণত অপারেটিং সিস্টেম যেমন লিনাক্স এবং xv6 এবং MS-DOS একটি নিরবিচ্ছিন্ন কার্ণামো ব্যবহার করে। সুতরাং, লিনাক্স অপারেটিং সিস্টেমের উদাহরণ বা লিনাক্স কার্নেলটির প্রায় 10 মিলিয়ন লাইন কোড আছে। সুতরাং, এই সমস্ত 10 মিলিয়ন লাইনের কোডটি সমগ্র লিনাক্স কার্নেলকে বোঝায় যা আসলে এই এলাকায় বিদ্যমান।

(স্লাইডটাইপ পড়ুন: 16:14)

অপারেটিং সিস্টেমের অন্য সাধারণ কার্ণামোটিকে মাইক্রোকার্নেল কার্ণামো বলা হয়, যেখানে কার্নেলটি অত্যন্ত উচ্চ মডিউল এবং কার্নেলের প্রত্যেকটি কম্পোনেন্টের তার নিজস্ব ঠিকানাযোগ্য স্থান থাকে। সুতরাং, এটি প্রতিটি

হিসাবে স্বাধীন প্রসেস যার জন্য একটি খুব ছোট মাইক্রোকার্নেল দরকার যা প্রকৃতপক্ষে কার্নেলে সংশ্লিষ্ট হয়, যা এই প্রক্রিয়াগুলির মধ্যে প্রতিটি যোগাযোগ ব্যবস্থাপনার দায়িত্বে, এবং ব্যবহারকারী প্রসেস এবং অপারেটিং সিস্টেমের মধ্যে যোগাযোগ প্রক্রিয়া। সুতরাং, এখানে সুবিধা এখানে এই মাইক্রোকার্নেল অত্যন্ত ছোট। তাই, আদর্শভাবে এটি আসলেই এল 1 ক্যাশ(L1 cache)এর মধ্যে ধরে যেতে পারে এবং এটি খুবই দ্রুত। সিস্টেম যাইহোক, সমস্যাটি হল যে আপনি একটি ডিভাইস ড্রাইভার ফাইল পরিচালনার থেকে সরাসরি কল করতে পারবেন না বা পরিবর্তে অকথ্য কার্নেলের মত না যেখানে আপনি একটি ফাইল পরিচালন মডিউল থেকে একটি ডিভাইস ড্রাইভার ফাংশন থেকে সরাসরি ফাংশন আহ্বান করতে পারেন। এখানে এই ফর্মের প্রতিটি আহ্বান একটি যোগাযোগ চ্যানেলের মাধ্যমে হওয়া উচিত যা আইপিসি বা ইন্টার প্রসেস কমিউনিকেশন চ্যানেল নামে পরিচিত। এই সাথে আমরা আসলে আজকের বক্তৃতা শেষ করছি, এবং পরবর্তী বক্তৃতা থেকে আমরা আসলে CPU সেয়ারিং সম্পর্কে আরও দেখব। ধন্যবাদ.